

# CS 101: Systems-Oriented Programming Fundamentals

An Exhaustive Rigorous Syllabus on Computational Models, Memory Abstractions & Algorithmic Mechanics

|                |                                            |              |                                             |
|----------------|--------------------------------------------|--------------|---------------------------------------------|
| Course Level:  | Undergraduate (Advanced Track / Core CS)   | Term / Year: | Academic Year 2026–2027                     |
| Prerequisites: | Discrete Mathematics & Basic Boolean Logic | Format:      | 4 Credit Hours (3h Lecture + 3h Lab / week) |

## 1. COURSE PHILOSOPHY & PEDAGOGICAL FRAMEWORK

This course provides a rigorous, foundational exploration of computing mechanics, moving beyond syntactical fluency to cultivate deep architectural mental models. Rather than treating programming languages as black-box abstractions, this curriculum grounds computational concepts in execution models, state transformation, memory layout, and resource management trade-offs.

Students will systematically analyze how high-level code maps down to machine instructions, evaluating the cognitive and performance trade-offs between static vs. dynamic typing, stack vs. heap allocation, manual memory lifecycle management, and control structure efficiency.

**Core Pedagogical Principle:** *"Understand the hardware underneath the abstraction."* Students are expected to reason explicitly about space-time complexity, cache locality, stack frames, and bit-level representations throughout all implementation exercises.

## 2. COMPREHENSIVE 16-WEEK CURRICULUM

| WEEK   | MODULE TOPIC                                                 | RIGOROUS CONTENT & THEORETICAL FOUNDATIONS                                                                                                                                                                                                        |
|--------|--------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Week 1 | Computing Abstractions & Execution Models <span>CORE</span>  | Von Neumann architecture; Register-Memory-ALU loop; Compilation pipeline (Preprocessing, Parsing, IR, Optimization, Code Gen, Linking) vs. Bytecode Interpretation vs. JIT; Memory Hierarchy overview.                                            |
| Week 2 | Representation of Data & Memory Layout <span>CORE</span>     | Primitive types, fixed-width integers, IEEE-754 floating-point format (sign, exponent, mantissa), Two's complement representation. Bitwise operations ( <code>AND</code> , <code>OR</code> , <code>XOR</code> , shifts), bit-masking, endianness. |
| Week 3 | Control Flow & Branch Prediction Mechanics <span>CORE</span> | Conditional evaluation, short-circuit boolean logic, state transitions, jump/branch instructions at the ISA level. Branch prediction impacts, switch table generation, jump tables vs. nested if-else statements.                                 |
| Week 4 | Iterative Dynamics & Loop Invariants <span>CORE</span>       | Bounded ( <code>for</code> ) vs Unbounded ( <code>while</code> ) loops. Proving correctness via Loop Invariants (Initialization, Maintenance, Termination). Loop unrolling performance trade-offs and induction variable elimination.             |
| Week 5 |                                                              |                                                                                                                                                                                                                                                   |

| WEEK    | MODULE TOPIC                                                                 | RIGOROUS CONTENT & THEORETICAL FOUNDATIONS                                                                                                                                                                                                                |
|---------|------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|         | <b>Functional Decomposition &amp; Call Stack Mechanics</b> <span>CORE</span> | Procedural abstraction, call stack framing (stack pointers, frame pointers, return addresses, parameter passing conventions). Scope lifetime, lexical scoping vs dynamic scoping, shadowing, reentrancy.                                                  |
| Week 6  | <b>Value vs. Reference Semantics &amp; Pointers</b> <span>ADVANCED</span>    | Pass-by-value, pass-by-reference, pass-by-pointer mechanics. Address-of operators, dereferencing, pointer arithmetic, null safety. Memory layout of stack-allocated variables vs memory addresses.                                                        |
| Week 7  | <b>Midterm Examination &amp; Synthesis</b> <span>EVAL</span>                 | Comprehensive practical and theoretical evaluation covering Weeks 1–6. Systems architecture written examination and low-level code diagnostic lab.                                                                                                        |
| Week 8  | <b>Contiguous Memory &amp; Array Subscripting</b> <span>CORE</span>          | Contiguous memory allocation, element offset calculations $\text{Address}(A[i]) = \text{Base} + i \times \text{Size}$ . Multi-dimensional array mapping (Row-major vs Column-major ordering), cache locality, spatial/temporal locality optimization.     |
| Week 9  | <b>Strings, Encodings &amp; Buffer Mechanics</b> <span>CORE</span>           | Null-terminated C-strings vs explicit length-prefixed strings. Character encodings (ASCII, UTF-8 variable length encoding, UTF-32). Buffer overflows, bounds checking, string immutability trade-offs.                                                    |
| Week 10 | <b>Compound Types, Structs &amp; Memory Alignment</b> <span>ADVANCED</span>  | User-defined composite types, structure padding, byte alignment requirements, memory layout visualization, struct packing optimization, tagged unions and variant types.                                                                                  |
| Week 11 | <b>Dynamic Memory Management &amp; The Heap</b> <span>ADVANCED</span>        | Heap allocation mechanics ( <code>malloc</code> / <code>free</code> or equivalent), allocation strategies (first-fit, best-fit, buddy allocator). Fragmentation (internal vs external), memory leaks, double frees, dangling pointers, Valgrind analysis. |
| Week 12 | <b>Algorithmic Foundations: Search &amp; Sort</b> <span>CORE</span>          | Linear vs Binary Search; Bubble, Insertion, Selection, and Merge Sort. Time and space complexity trade-offs. Comparative analysis of adaptive vs non-adaptive sorting algorithms.                                                                         |
| Week 13 | <b>File I/O, Persistence &amp; Stream Abstractions</b> <span>APPLIED</span>  | File descriptors, stream buffering (unbuffered, line-buffered, fully-buffered). Binary vs Text file I/O, serialization and deserialization protocols, file position pointers, error state flags.                                                          |
| Week 14 | <b>Defensive Programming &amp; Exception Handling</b> <span>APPLIED</span>   | Preconditions, postconditions, and invariant assertions ( <code>assert</code> ). Error propagation models: Return codes vs Exception handling stack unwinding. Resource Acquisition Is Initialization (RAII) principles.                                  |
| Week 15 | <b>Asymptotic Analysis &amp; Fundamental ADTs</b> <span>ADVANCED</span>      | Formal Big-O ( $\mathcal{O}$ ), Big-Omega ( $\Omega$ ), and Big-Theta ( $\Theta$ ) notation. Implementation of Abstract Data Types: Stacks (LIFO), Queues (FIFO), and Single/Doubly Linked Lists from scratch.                                            |
| Week 16 | <b>Capstones Defense &amp; System Code Synthesis</b> <span>APPLIED</span>    | Final project code defense, static analysis audit, profiling memory usage, peer code review, comprehensive final written examination.                                                                                                                     |

### 3. ASSESSMENT & GRADING FRAMEWORK

Evaluation is structured to emphasize both mathematical/theoretical rigour and low-level software execution proficiency.

| Component                         | Weight | Description & Objectives                                                                                                          |
|-----------------------------------|--------|-----------------------------------------------------------------------------------------------------------------------------------|
| <b>Weekly Programming Labs</b>    | 25%    | Hands-on implementation of low-level algorithms, bit-manipulation tasks, dynamic memory structures, and custom data parsers.      |
| <b>Problem Sets (Theoretical)</b> | 15%    | Mathematical proofs of loop invariants, manual trace matrices, memory layout diagrams, and Big-O derivations.                     |
| <b>Midterm Examination</b>        | 20%    | Comprehensive evaluation of Weeks 1–6: bitwise math, execution stack analysis, control flow tracing.                              |
| <b>Systems Capstone Project</b>   | 15%    | End-to-end software module in C/C++ or Rust (e.g., custom memory allocator, custom shell, or binary file parser) with zero leaks. |
| <b>Final Examination</b>          | 25%    | Cumulative examination assessing full-spectrum architectural understanding, complexity analysis, and defensive program design.    |

## 4. RECOMMENDED LITERATURE & REFERENCE MATERIALS

---

- Kernighan, B. W., & Ritchie, D. M. *The C Programming Language* (2nd Ed.). Prentice Hall.
- Bryant, R. E., & O'Hallaron, D. R. *Computer Systems: A Programmer's Perspective* (3rd Ed.). Pearson.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. *Introduction to Algorithms* (4th Ed.). MIT Press.
- Sedgewick, R., & Wayne, K. *Algorithms* (4th Ed.). Addison-Wesley Professional.